Université Lumières Lyon 2
UFR de Sciences Economiques et de Gestion
Master 1 MBFA - Monnaie, Banque Finance, Assurance

Gestion d'une galerie d'art

Evaluation d'Initiation à SQL

Réalisé par :
FRAU Christophe
BENSOUSSAN Ruben
TD 1

I - Introduction

1. Contexte général du projet

Dans le cadre de l'évaluation du module d'Initiation à SQL du Master 1 MBFA, ce projet vise à concevoir et implémenter une base de données complète à partir d'un cas concret du monde réel.

L'objectif est de mettre en pratique les compétences acquises en modélisation conceptuelle et logique, ainsi qu'en implémentation SQL sous PostgreSQL, à travers une problématique de gestion d'informations complexes.

2. Choix et justification de la problématique

Le domaine choisi est celui de la gestion d'une galerie d'art car cette problématique présente un environnement où de nombreuses données doivent être suivies tel que cela a pu être constaté par M. FRAU dans son expérience professionnelle de directeur de galerie d'art. De plus, il s'agit d'un domaine où la manipulation et le suivi de l'information est primordial.

En effet, dans une galerie d'art, de nombreuses entités interagissent : les artistes créent des œuvres, ces œuvres peuvent exister dans diverses éditions. Ces œuvres sont également vendues et présentées lors d'expositions.

Pour les galeries d'art, la principale difficulté rencontrée est la structuration et la centralisation des informations Ainsi, nous allons essayer de concevoir une base de données permettant une gestion efficace tout en garantissant la cohérence et la traçabilité des informations.

Dans la perspective d'un projet SQL, ce thème a été retenu pour plusieurs raisons :

- Illustrer de façon naturelle la notion de relations entre entités
- Permettre d'introduire des contraintes métiers claires
- Présenter un équilibre entre complexité et réalisme

3. Identification des besoins

Pour répondre à cette problématique, la base de donnée doit répondre aux besoins suivants :

- Gérer les informations relatives aux artistes
- Répertorier les œuvres créées par chaque artiste, avec leurs caractéristiques
- Suivre les éditions physiques de chaque œuvre
- Organiser les expositions, avec leurs caractéristiques.
- Gérer la clientèle et leurs coordonnées, tout en suivant leur engagement
- Tracer les transactions pour garantir un historique commercial complet.

4. Contraintes spécifiques

Afin que la base de données soit fonctionnelle et réaliste, il est nécessaire de prendre en compte plusieurs contraintes :

- Intégrité des données : les liens entre artistes, œuvres, éditions et transactions doivent être cohérents.
- Suivi du statut des éditions : chaque édition peut être *disponible*, *réservée* ou *vendue*, et ce statut doit se mettre à jour en fonction des transactions.
- Gestion temporelle : les expositions et les transactions doivent inclure des dates précises pour permettre un suivi chronologique.
- Éviter la redondance d'information : les données communes (ex. nom d'artiste ou titre d'œuvre) doivent être centralisées.

II - Modélisation conceptuelle

La modélisation conceptuelle va nous permettre de représenter les informations essentielles du système. L'objectif est de comprendre et formaliser les besoins fonctionnels exprimés dans la problématique, tout en identifiant :

- Les entités principales
- Les relations entre ces entités
- Les contraintes afin de garantir la cohérence du système

1. Présentation générale

Le modèle repose sur 6 entités principales :

- Artiste
- Oeuvre
- Edition
- Exposition
- Client
- Transaction

Ces entités sont reliées entre elles par des relations permettant de représenter les interactions naturelles propres au fonctionnement d'une galerie.

2. Description détaillée des entités et relations

L'entité Artiste, qui représente les créateurs exposant leurs œuvres au sein de la galerie comprend les attributs suivants : identifiant (clé primaire), nom, prénom, date_naissance, pays. Un artiste peut être associé à plusieurs œuvres.

L’entité œuvre représente la création unique d’un artiste. Elle comprend les attributs suivants : identifiant (clé primaire), titre, identifiant de l’artiste, année de création, le type de support et la description. De plus, chaque œuvre est créée par un seul artiste et chaque œuvre peut avoir plusieurs éditions.

L’entité Édition correspond à une édition physique d’une œuvre. Elle comprend les attributs : identifiant (clé primaire), identifiant physique de l'œuvre, tirage, dimension, prix, le statut (disponible, réservée, vendue) et la date d’acquisition. Une édition appartient à une œuvre et est concernée par une unique transaction. Enfin, une édition peut être associée à plusieurs expositions.

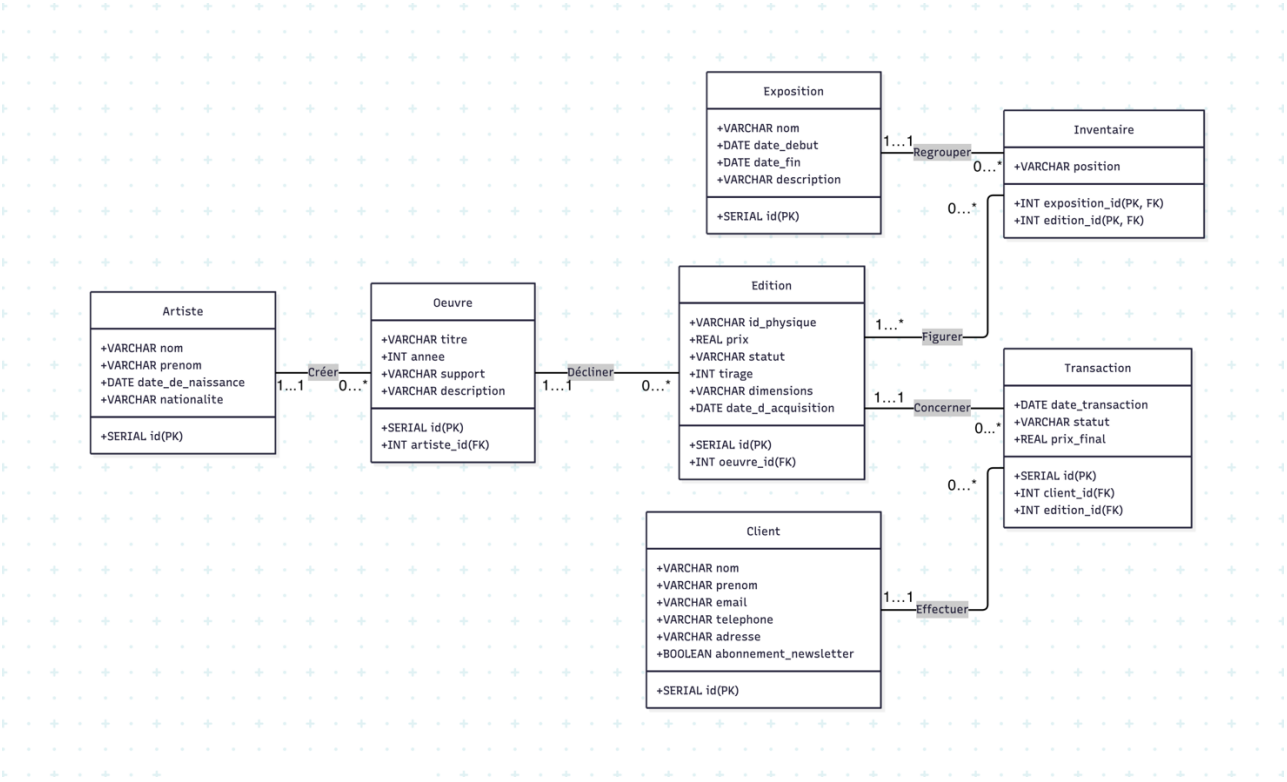
L’entité Exposition représente un événement organisé par la galerie. Elle comprend les attributs identifiants (clé primaire), le nom de l’exposition, la daté de début, la date de fin et la description. Une exposition peut présenter plusieurs expositions.

L’entité Inventaire est une table d’association et représente les expositions et éditions exposées. Elle comprend les attributs : identifiant de l’édition, identifiant de l’exposition et la position dans l’exposition.

L’entité Client représente les acheteurs de la galerie et comprend les attributs : identifiant (clé primaire), nom, prénom, mail, téléphone, adresse et si le client est abonné à la newsletter. Un client peut effectuer plusieurs transactions.

L’entité Transaction enregistre l'état des transactions entre la galerie et les clients et comprend les attributs : identifiant (clé primaire), identifiant de l’édition, l’identifiant du client, le statut de la transaction (complétée, annulée, en attente), la date et le prix. Chaque transaction est reliée à un client et une édition et une édition peut être vendue plusieurs fois (si elle existe en plusieurs exemplaires).

3. <u>Schéma conceptuel - UML</u>



Le modèle conceptuel répond aux besoins identifiés. Il permet ainsi de gérer la vie d'une œuvre depuis sa création jusqu'à sa vente. Il intègre aussi les expositions, essentielles au fonctionnement et à la vie d'une galerie. De plus, le modèle garantit la traçabilité des éditions des différentes œuvres ainsi que des transactions.

III - Modélisation logique

1. Schéma relationnel

Nous allons désormais traduire le modèle conceptuel établi précédemment en modèle relationnel.

ARTISTE(**Id**, Nom, Prenom, Date_Naissance, Pays)

ŒUVRE(**Id**, Titre, Annee, Support, Description, Id_Artiste#)

ÉDITION(**Id**, Id_Physique, Tirage, Dimensions, Prix, Statut, Date_Acquisition, Id_Oeuvre#)

EXPOSITION(**Id**, Nom, Date_Debut, Date_Fin, Description)

CLIENT(**Id**, Nom, Prenom, Mail, Telephone, Adresse, Abonnement_Newsletter)

TRANSACTION(**Id**, Statut, Date, Prix, Id_Edition#, Id_Client#)

INVENTAIRE(**Id_Edition#**, **Id_Exposition#**, Position)

2. Justification des choix de modélisation

Les entités Artiste, Oeuvre, Client et Exposition sont autonomes et existent indépendamment des autres données. On utilise pour chaque entité une clé primaire, représentée ici en gras. Les tables Inventaire et Transaction correspondent à des associations. La table Inventaire permet de faire le lien entre les entités Édition et Exposition car cette relation nécessite une table propre afin de pouvoir stocker la position de chaque œuvre dans l'exposition. La table Transaction relie les entités Edition et Client permet d'avoir un suivi du statut ainsi que de garder un historique des transactions, des prix et des statuts des transactions. Enfin, les clés étrangères permettent et garantissent l'intégrité référentielle entre les tables.

Chaque entité correspond à une table SQL et chaque association comportant les cardinalités N…N, représenté sur le graphique par *, devient alors une table associative. De plus, les tables avec les cardinalités N dans le modèle conceptuel se traduisent par une clé étrangère du côté de la table avec la cardinalité N. Par exemple, la relation Artiste - Oeuvre a une relation avec une cardinalité 0…N, ce qui se traduit par l'ajout de l'identifiant Artiste dans la table Oeuvre.

IV - Implémentation SQL

1. Présentation

A l'aide du logiciel DBeaver, nous allons désormais pouvoir implémenter la base de données en langage PostgreSQL. L'objectif est de représenter le modèle relationnel tout en respectant l'ensemble des contraintes d'intégrité référentielle, les types de données et avoir une cohérence au niveau des relations entre les tables.

Comme vu à la fin de la partie précédente, chaque table correspond à une entité ou une relation du modèle conceptuel. Nous allons donc présenter les scripts SQL utilisés pour créer les tables, remplir ces tables de données et les requêtes représentatives utilisées.

2. Création des tables

```
CREATE TABLE Artiste (id SERIAL PRIMARY KEY, nom VARCHAR(100) NOT NULL, prenom VARCHAR(100), date_naissance DATE, pays VARCHAR(100));

CREATE TABLE Oeuvre (id SERIAL PRIMARY KEY, titre VARCHAR(200) NOT NULL, artiste_id INT REFERENCES Artiste(id) ON DELETE CASCADE, annee INT, support VARCHAR(100), description TEXT);

CREATE TABLE Edition (id SERIAL PRIMARY KEY, oeuvre_id INT REFERENCES Oeuvre(id), id_physique VARCHAR(20), tirage INT, dimensions VARCHAR(50), prix REAL, statut VARCHAR(20) CHECK (statut IN ('disponible','reservé','vendu')), date_acquisition DATE);

CREATE TABLE Exposition (id SERIAL PRIMARY KEY, nom VARCHAR(200) NOT NULL, date_debut DATE, date_fin DATE, description TEXT);

CREATE TABLE Inventaire (id SERIAL PRIMARY KEY, exposition_id INT REFERENCES Exposition(id), edition_id INT REFERENCES Edition(id), position VARCHAR(50));

CREATE TABLE Client (id SERIAL PRIMARY KEY, nom VARCHAR(100), prenom VARCHAR(100), mail VARCHAR(150) UNIQUE, telephone VARCHAR(50), adresse TEXT, abonnement_newsletter BOOLEAN DEFAULT FALSE);

CREATE TABLE Transaction (id SERIAL PRIMARY KEY, edition_id INT REFERENCES Edition(id), client_id INT REFERENCES Client(id), statut VARCHAR(20) CHECK (statut IN ('en attente','complétée','annulée')), date DATE, prix REAL);
```

La mention PRIMARY KEY correspond à l'attribution de la clé primaire de la table. La mention REFERENCES suivi du titre d'une autre table correspond à l'attribution d'une clé étrangère. Le terme VARCHAR(n) permet que l'attribut prend la forme du chaîne de caractères de maximum n caractères. DATE correspond à une date au format 'YYYY-MM–DD', soit année-mois-jour. INT permet d'attribuer un nombre entier. Enfin, ON DELETE CASCADE permet d'assurer la cohérence des données lorsqu'une table contient une clé étrangère, en supprimant la ligne de la table si la ligne correspondante dans la table d'où provient la clé étrangère est également supprimée.

3. Insertion de données d'exemple

```
-- Artistes
INSERT INTO Artiste (nom, prenom, date_naissance, pays) VALUES ('Picasso', 'Pablo', '1881-10-25', 'Espagne'), ('Monet', 'Claude', '1840-11-14', 'France'), ('Rodin', 'Auguste', '1840-11-12', 'France');

INSERT INTO Oeuvre (titre, artiste_id, annee, support, description) VALUES ('Les Demoiselles d'Avignon', 1, 1907, 'Huile sur toile', 'Icône du cubisme.'), ('Nymphéas', 2, 1916, 'Huile sur toile', 'Série impressionniste célèbre.'), ('Le Penseur', 3, 1902, 'Bronze', 'Sculpture emblématique.');

INSERT INTO Edition (oeuvre_id, id_physique, tirage, dimensions, prix, statut, date_acquisition) VALUES (1, 'A001', 10, '100x80', 2000000, 'vendu', '2020-01-10'), (2, 'M001', 5, '120x90', 1500000, 'disponible', '2021-06-15'), (3, 'R001', 3, '70x50', 900000, 'reservé', '2019-09-20');

INSERT INTO Exposition (nom, date_debut, date_fin, description) VALUES
('Impressionnisme et Modernité', '2023-03-01', '2023-06-01', 'Dialogue entre Monet et Picasso.'), ('Sculpter le Mouvement', '2024-04-15', '2024-09-15', 'Focus sur Rodin.');

INSERT INTO Inventaire (exposition_id, edition_id, position) VALUES (1, 1, 'Salle 1'), (1, 2, 'Salle 2'), (2, 3, 'Salle 3');

INSERT INTO Client (nom, prenom, mail, telephone, adresse, abonnement_newsletter) VALUES ('Dupont', 'Alice', 'alice.dupont@mail.com', '0601020304', '12 rue des Fleurs, Paris', TRUE), ('Martin', 'Louis', 'louis.martin@mail.com', '0605060708', '5 avenue des Champs, Lyon', FALSE), ('Durand', 'Emma', 'emma.durand@mail.com', '0610101010', '3 boulevard Victor Hugo, Nice', TRUE);

INSERT INTO Transaction (edition_id, client_id, statut, date, prix) VALUES (1, 1, 'complétée', '2021-01-20', 2000000), (2, 2, 'en attente', '2024-02-12', 1500000), (3, 3, 'annulée', '2023-11-30', 900000);
```

4. Requêtes SQL

Les requêtes permettent de répondre aux besoins fonctionnels de la galerie d'art. Nous en avons effectués plusieurs afin de voir l'ensemble des requêtes possibles

Requête 1 : Lister tous les artistes dont le pays est la France

```
SELECT nom, prenom, pays
FROM Artiste
WHERE pays = 'France';
```

SELECT permet de sélectionner les attributs qui nous intéressent, FROM permet de définir la table d'où proviennent ces informations et WHERE permet d'ajouter une condition. LEs résultats attendus sont : Monet, Claude, France et Rodin, Auguste, France

Requête 2 : Afficher chaque oeuvre avec le nom de son artiste

```
SELECT o.titre, a.nom
FROM Oeuvre o
JOIN Artiste a ON o.artiste_id = a.id;
```

JOIN permet d'effectuer une jointure entre deux tables lorsque les informations se trouvent dans deux tables distinctes. Le résultat attendu est : Les Demoiselles d'Avignon, Picasso; Nymphéas, Monet; Le Penseur, Rodin.

Requête 3 : Compter le nombre d'oeuvres de que chaque artiste a créé

```
SELECT a.nom, COUNT(o.id) AS nb_oeuvres
FROM Artiste a
LEFT JOIN Oeuvre o ON a.id = o.artiste_id
GROUP BY a.nom;
```

LEFT JOIN permet de conserver l'entièreté des artistes même si ceux ci n'ont aucune œuvre. GROUP BY permet de regrouper les résultats par artistes afin d'effectuer un comptage. Les résultats attendus sont : Monnet, 1; Rodin, 1; Picasso, 1.

Requête 4 : Lister les transactions réalisées avec le client, l'artiste, l'oeuvre et le prix

```
SELECT t.id, c.nom AS client, a.nom AS artiste, o.titre, t.prix, t.statut
FROM Transaction t
JOIN Client c ON t.client_id = c.id
JOIN Edition ed ON t.edition_id = ed.id
JOIN Oeuvre o ON ed.oeuvre_id = o.id
JOIN Artiste a ON o.artiste_id = a.id;
```

Cette requête est un exemple plus poussé de la jointure, expliquée précédemment. Les résultats attendus sont : Dupont, Picasso, Les Demoiselles d'Avignon, 2000000, complétée; Martin, Monet, Nymphéas, 1500000, en attente; Leroy, Rodin, Le Penseur, 900000, annulée.

Requête 5 : Lister toutes les oeuvres dont la moyenne des éditions dépassent 1 000 000€

```
SELECT titre
FROM Oeuvre
WHERE id IN (
  SELECT oeuvre_id
  FROM Edition
  GROUP BY oeuvre_id
  HAVING AVG(prix) > 1000000 );
```

Cette requête comprend une sous requête permettant de rassembler les œuvres dont le prix moyen dépasse le million d'euros et de les regrouper. Les résultats attendus sont les Demoiselles d'Avignon et Nymphéas.

Requête 6 : Afficher les éditions qui n'ont pas encore été vendues

```sql
SELECT id_physique, prix
FROM Edition
WHERE id NOT IN (
  SELECT edition_id
  FROM Transaction
  WHERE statut = 'complétée');
```

La sous-requête permet de sélectionner les œuvres dont la transaction a été complétée, ce qui signifie qu'elles ont été vendues. WHERE…NOT IN permet de sélectionner les œuvres ne se trouvant pas dans sélection afin de répondre à la requête. Les résultats attendus sont : M001, 1 500 000; R001, 900 000.

Requête 7 : Artistes dont toutes les oeuvres datent d'avant 1950

```sql
SELECT a.nom
FROM Artiste a
WHERE NOT EXISTS (
  SELECT *
  FROM Oeuvre o
  WHERE o.artiste_id = a.id
  AND o.annee >= 1950);
```

WHERE…NOT EXISTS permet de vérifier qu'il n'existe pas d'œuvre postérieure à 1950 pour chaque artiste. La sous-requête permet de chercher les artistes disposant d'œuvres après 1950 et si aucun n'est trouvé alors l'artiste est retenu. Les résultats attendus sont : Picasso; Monet; Rodin.

Requête 8 : Expositions où au moins une édition exposée a été vendue

```sql
SELECT DISTINCT ex.nom
FROM Exposition ex
WHERE EXISTS (
  SELECT *
  FROM Inventaire i
  JOIN Edition e ON i.edition_id = e.id
  JOIN Transaction t ON e.id = t.edition_id
  WHERE i.exposition_id = ex.id AND t.statut = 'complétée'
```

Cette requête reprend les éléments vus précédemment et fait apparaître DISTINCT qui permet d'éviter les doublons, les répétitions. Le résultat attendu est Impressionnisme et Modernité.

Requête 9 : Éditions plus chères que toutes celles de Monet

```sql
SELECT e.id_physique, e.prix
FROM Edition e
WHERE e.prix > ALL (
  SELECT ed.prix
  FROM Edition ed
```

```
JOIN Oeuvre o ON ed.oeuvre_id = o.id
JOIN Artiste a ON o.artiste_id = a.id
WHERE a.nom = 'Monet');
```

La partie WHERE … > ALL permet de comparer chaque édition à l'entièreté des prix des éditions de Monet, obtenus par la sous-requête. Le résultat attendu est A001, 2 000 000.